SAMSUNG

Samsung Innovation Campus

Generative AI Module

Together for Tomorrow!
Enabling People
Education for Future Generations

Notas de Clase

Generative AI Module

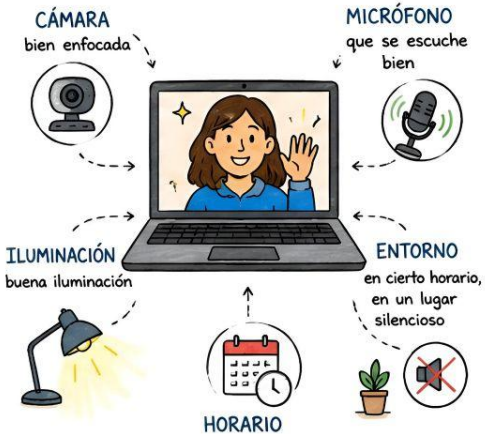
OBJETIVOS DEL CURSO

1 Se lleven como razonar un problema y saber cuando usar AI y cuando no.



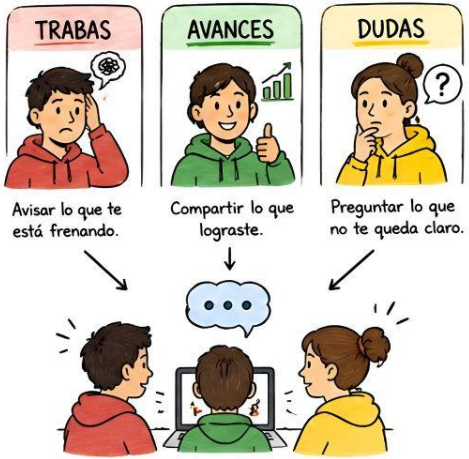
Primero pensar, después decidir.

2 Organizar su vida para tener meets, eso implica tener tu carta de presentacion en buenas condiciones:



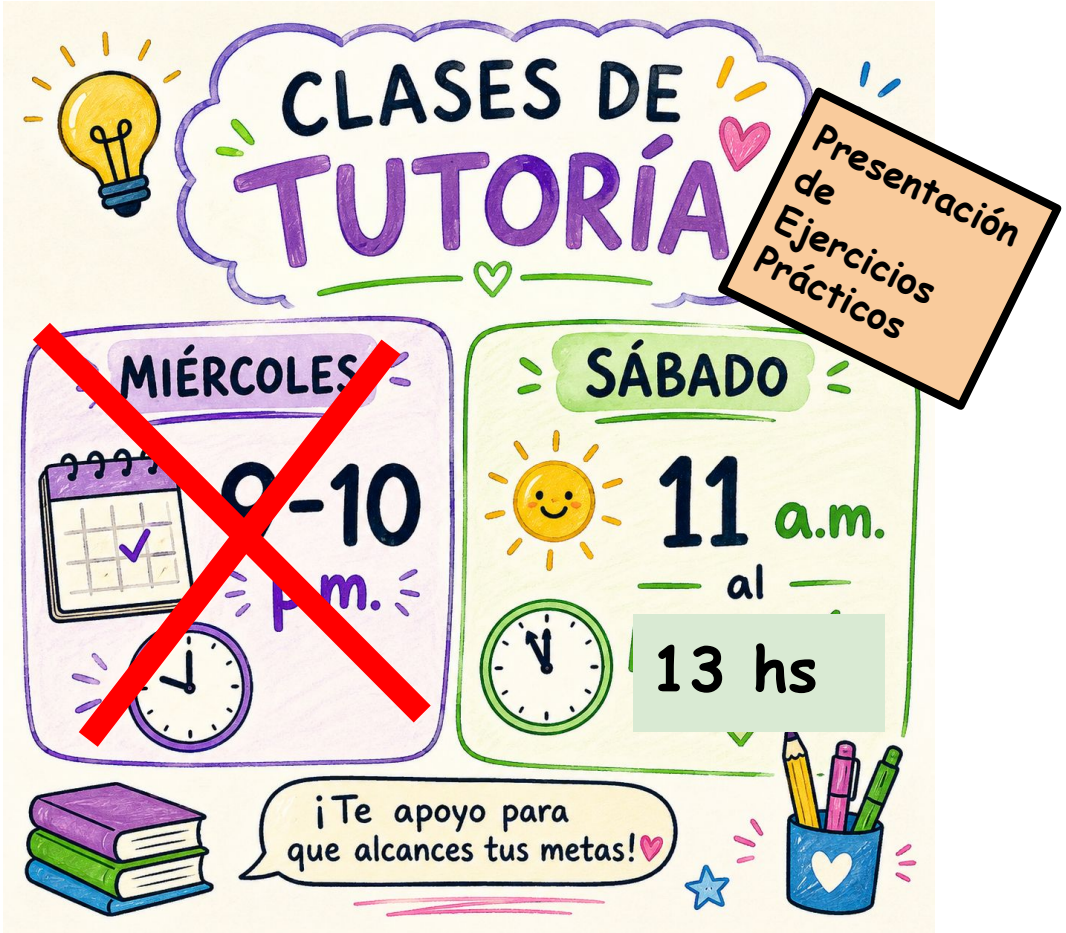
Tu meet es tu carta de presentación. ¡Cuidala!

3 Poder comunicar trabas, avances y dudas a tiempo, es preferible equivocarse aca que con una deadline.



Comunicar a tiempo = mejores soluciones, menos estrés.

PENSAR. ORGANIZAR. COMUNICAR. = CRECER JUNTOS



[Inscripción a la reunión - Zoom]

CLASE ANTERIOR

Evaluación: Quiz de NumPy

QUIZ NUMPY: FUNCIONES, VECTORES Y MATRICES

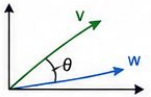
1 FUNCIONES IMPORTANTES

- Creación de arrays: `np.array()`, `np.zeros()`, `np.ones()`, `np.arange()`, `np.linspace()`
- Operaciones: `+`, `-`, `*`, `/` (elemento a elemento), `np.dot()` (producto punto), `@` (producto matricial)
- Estadísticas: `np.sum()`, `np.mean()`, `np.min()`, `np.max()`, `np.std()`

NumPy

2 VECTORES

- Creación: `v = np.array([1, 2, 3])`
- Norma (euclídea): `np.linalg.norm(v)` = $\sqrt{(v_2^2 + v_2^2 + \ldots + v_n^2)}$
- Distancia entre vectores: `np.linalg.norm(v − w)`
- Producto punto: $v \cdot w = \sum_i v_i w_i$



$v \cdot w = ||v||\ ||w|| \cos\theta$

3 MATRICES

- Creación: `A = np.array([[1, 2], [3, 4], [5, 6]])`
- Forma (shape): `A.shape` → (3, 2)
- Transpuesta: `A.T`
- Determinante: `np.linalg.det(A)`
- Inversa: `np.linalg.inv(A)`
- Producto matricial: `A @ B`
- Identidades importantes: I_n, $A\ A^{-1} = I$

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

$$\det(A) = -2$$

$$A^{-1} = \frac{1}{-2} \cdot \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix}$$

TIPS PARA EL QUIZ

- ✓ Revisá las formas (shape)
- ✓ Cuidá las dimensiones
- ✓ Producto punto ≠ Producto matricial
- ✓ Usá funciones de NumPy (evitá bucles)

EJEMPLO RÁPIDO

```
A = np.array([[1,2,3],[4,5,6]])
B = np.array([[1],[0],[1]])
A @ B   # resultado: [[4],[10]]
```

¡Excelente trabajo! Practicamos, medimos y consolidamos lo aprendido.

CLASE DE HOY

VISIÓN GENERAL



1. APIFY – LA PLATAFORMA

- Plataforma en la nube para ejecutar y gestionar automatizaciones, scraping y procesamiento de datos.
- Infraestructura escalable, segura y gestionada.
- Provee herramientas, UI, API, almacenamiento y monitoreo.

2. ACTOR – QUÉ ES

- Un Actor es un programa listo para realizar una tarea específica.
- Ejemplos: Web Scraper, PDF Text Extractor, Image Scraper, Google Search, etc.
- Cada Actor define el input que recibe y el output que produce.

3. RUN – UNA EJECUCIÓN

- Cuando ejecutás un Actor, Apify crea un Run.
- Cada Run tiene: input, estado (RUNNING / SUCCEEDED / FAILED), duración, consumo de recursos y output.
- Un mismo Actor puede tener muchos Runs.

4. ENDPOINT – LA PUERTA DE ENTRADA

- Es la URL (endpoint) que permite comunicarse con Apify mediante HTTP.
- Ejemplo de endpoint para ejecutar y obtener resultados al instante:

```
POST /v2/actors/{actorId}/run-sync-get-dataset-items
```

- Se envía el input (JSON) y se obtienen los resultados (dataset items).

5. CLIENTE – QUIEN REALIZA LA PETICIÓN

- Puede ser Make, Python, JavaScript/Node.js, cURL, Postman o tu propia aplicación.
- El cliente envía la petición al endpoint con el input necesario.
- Recibe el output y lo usa en su flujo o aplicación.

RESUMEN DE LA ARQUITECTURA

Cliente →	Endpoint (API) →	Actor →	Run →	Output (Datos)
Quien llama	Puerta de entrada	Programa / Herramienta	Ejecución	Resultados

TIP CLAVE

- Actor = programa
- Run = ejecución
- Endpoint = URL/API
- Cliente = quien llama
- Output = resultados

PANDAS

GUÍA VISUAL PARA ENTENDER Y ANALIZAR DATOS TABULARES

DE DATOS CRUDOS A INFORMACIÓN

DATOS → PANDAS → ANÁLISIS → DECISIONES

1 ¿PARA QUÉ SIRVE PANDAS?

NumPy
cálculo numérico sobre arrays (datos homogéneos) → **pandas**
datos tabulares y **heterogéneos**

Estructura central: DataFrame
Una columna de un DataFrame es una Series

2 DATAFRAME = TABLA

	nombre	edad	ciudad	salario
0	Ana	20	CABA	50000
1	Juan	25	La Plata	60000
2	Pedro	31	Córdoba	70000

INDICE → FILAS = observaciones

COLUMNAS = variables

Cada columna es una Series

3 SELECCIÓN DE DATOS

Por columnas

```
df["edad"]
  # devuelve una Series
df[["edad", "nombre"]]
  # devuelve un DataFrame
```

Por etiquetas (loc)

```
df.loc[1]
  # fila con índice 1
df.loc[0:2, ["nombre", "salario"]]
  # filas 0 a 2, columnas elegidas
```

Por posición (iloc)

```
df.iloc[1]
  # segunda fila
df.iloc[0:2, 1:3]
  # filas 0 a 2, columnas
    1 a 3 (por posición)
```

Info útil:
`df.index` → índices (etiquetas de filas)
`df.columns` → nombres de columnas

4 OPERACIONES VECTORIZADAS Y ALINEACIÓN POR ÍNDICE

Vectorizado (rápido)

```
df["edad"] + 10
```

Opera elemento a elemento

Alineación por índice

Serie A	
Ana	20
Juan	25
Pedro	31

+

Serie B	
Pedro	100
Ana	200
Lucía	300

Resultado (A + B)

Ana	220
Juan	NaN
Pedro	131
Lucía	NaN

Pandas alinea por índice. Si no hay coincidencia → NaN

5 FILTRADO (CONDICIONES BOOLEANAS)

```
df[df["edad"] > 25]
```

Solo pasan los que cumplen la condición

Ejemplo:

nombre	edad	ciudad
Ana	20	CABA
Juan	25	La Plata
Pedro	31	Córdoba
Lucía	28	CABA

`df[df["edad"] > 25]`

Pedro	31	Córdoba
Lucía	28	CABA

6 COMBINAR DATOS: MERGE (JOIN)

TABLA A

id	nombre
1	Ana
2	Juan
3	Pedro

TABLA B

id	salario
1	500
3	800
4	900

INNER JOIN	Solo los IDs que están en ambas tablas
LEFT JOIN	Todas las filas de A + coincidencias de B
RIGHT JOIN	Todas las filas de B + coincidencias de A
OUTER JOIN	Todas las filas de A y todas las de B

7 AGRUPAR Y AGREGAR (GROUPBY)

FILTRAR (seleccionar datos) → AGRUPAR (por una o más columnas) → AGREGAR (funciones de resumen) → RESULTADO (nuevo DataFrame)

Ejemplo:

```
df.groupby
("ciudad")["salario"].mean()
```

ciudad	salario_promedio
CABA	55000
Córdoba	70000
La Plata	60000

8 ESTADÍSTICA DESCRIPTIVA

¿Qué pregunta responde cada medida?

- **count** → ¿cuántos datos tengo?
- **mean** → ¿cuál es el promedio?
- **std** → ¿qué tan dispersos están?
- **min** → ¿cuál es el menor?
- **max** → ¿cuál es el mayor?

`df.describe()`

count	3.0
mean	25.3
std	5.5
min	20.0
25%	22.5
50%	25.0
75%	28.0
max	31.0

Resumen rápido de una columna numérica

9 NaN: VALORES FALTANTES

¿Qué es NaN?
NaN = falta / ausencia de un valor válido

NaN ≠ 0
NaN ≠ "dato desconocido = cero"

¿Dónde aparece?

- Merge / Join
- Operaciones entre Series
- Filtrado
- Estadísticas
- Groupby
- Datos faltantes

¿Qué puedo hacer?

- Detectar → `df.isna()`
- Contar → `df.isna().sum()`
- Proporción → `df.isna().mean()`
- Rellenar → `df.fillna(...)`
- Eliminar → `df.dropna(...)`

10 NIVELES DE PRÁCTICA (COLAB)

NIVEL 1 BÁSICO
- ✓ DataFrame
- ✓ Selección de columnas y filas
- ✓ Filtros (condiciones booleanas)
- ✓ Información básica (index, columns)

NIVEL 2 INTERMEDIO
- ✓ Transformaciones
- ✓ Groupby + agregaciones
- ✓ Estadísticas descriptivas
- ✓ Funciones de resumen (mean, sum, count...)

NIVEL 3 AVANZADO
- ✓ Merge / Join
- ✓ NaN: detectar, contar, rellenar, eliminar
- ✓ Análisis combinado de múltiples tablas
- ✓ Reportes y conclusiones

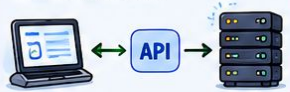
Piensa en los sistemas como personas que hablan

INTEGRACIONES Y FLUJOS DE DATOS:

Cómo se comunican los sistemas y cómo viajan los datos

¿QUÉ ES UNA API?

Una API (Application Programming Interface) es un contrato que permite que dos sistemas se comuniquen e intercambien datos.



La API define qué pedir (endpoints), cómo hacerlo (métodos) y qué formato de respuesta se obtiene.

¿QUÉ ES UN SERVER?

Un servidor es una computadora o programa que almacena datos, ejecuta aplicaciones y responde a solicitudes a través de la red (internet o privada).



Un servidor "escucha" en una dirección (IP) y puerto específicos y espera solicitudes.

¿QUÉ ES CRUD?

Las 4 operaciones básicas sobre los datos.

Operación	Significado	Ejemplo
C	Create (Crear)	Crear un registro nuevo
R	Read (Leer)	Obtener o consultar registros
U	Update (Actualizar)	Modificar un registro existente
D	Delete (Eliminar)	Eliminar un registro

CRUD aplica a muchos tipos de datos: usuarios, productos, pedidos, archivos, etc.

CLIENTE vs ENDPOINT

CLIENTE

Quien inicia la comunicación.



Puede ser: app, navegador, script, servicio, etc.

ENDPOINT

La dirección específica (URL) de la API que recibe la solicitud y devuelve una respuesta.

https://api.ejemplo.com/usuarios

Cada endpoint expone una función o recurso del servidor (ej.: /usuarios).

¿POR QUÉ LAS APIS USAN JSON?

JSON (JavaScript Object Notation) es un formato de texto liviano para intercambiar datos.

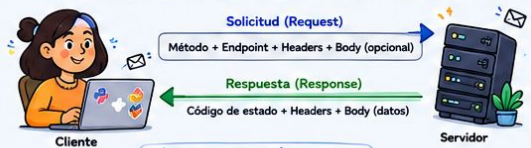
- Liviano y legible.
- Independiente del lenguaje.
- Estructurado (objetos y listas).
- Fácil de generar y parsear.
- Ampliamente soportado.
- Ideal para la web y servicios.

```
{
  "id": 1,
  "nombre": "Ana",
  "email": "ana@
   ejemplo.com",
  "activo": true,
  "roles": ["admin",
            "editor"]
}
```

JSON es solo un formato de datos. También existen XML, CSV, Protocol Buffers, etc.

¿CÓMO SE COMUNICAN?

La comunicación entre cliente y servidor ocurre usando protocolos y mensajes.



MÉTODOS HTTP MÁS COMUNES

GET	POST	PUT	PATCH	DELETE
Consultar datos (Idempotente)	Crear datos (No idempotente)	Actualizar todo un recurso	Actualizar parte de un recurso	Eliminar un recurso

¿ES UN PIPELINE?

Sí. Un flujo de datos sigue una secuencia de pasos:



Ejemplos de pipeline



Un pipeline puede ser manual (vos lo hacés) o automático (se ejecuta solo).

EJEMPLO DE FLUJO (PIPELINE)



Los datos entran → se procesan → se entregan como resultado.
Así funcionan las integraciones y automatizaciones.

EN RESUMEN

- Las APIs permiten que los sistemas se comuniquen de forma estandarizada.
- Un server ofrece servicios a través de endpoints.
- CRUD son las operaciones básicas sobre los datos.
- Los flujos (pipelines) automatizan: entrada → transformación → salida.
- JSON es el formato más usado para intercambiar datos en APIs.

GLOSARIO RÁPIDO

- **API:** Interfaz para conectar sistemas.
- **Server:** Provee recursos/servicios.
- **Cliente:** Consume esos servicios.
- **Endpoint:** URL que expone un recurso.
- **JSON:** Formato de intercambio de datos.
- **Pipeline:** Secuencia de pasos que transforman datos.

VISIÓN COMPLETA (TODO JUNTO)



Protocolo: HTTP / HTTPS

EJEMPLO DE ENDPOINT (REAL)

```
GET https://api.ejemplo.com/usuarios

Headers:
  Authorization: Bearer TOKEN
  Accept: application/json
Respuesta:
  [
    { "id": 1,  "nombre": "Ana" },
    { "id": 2,  "nombre": "Luis" }
  ]
```

CRUD vs SQL

¿En qué se diferencian y cómo se relacionan?

CRUD

ES UN MODELO / CONCEPTO

Describe las operaciones básicas que podemos hacer con los datos.

- **C** — **CREATE** (Crear): Crear un nuevo registro
- **R** — **READ** (Leer): Obtener o consultar registros
- **U** — **UPDATE** (Actualizar): Modificar un registro existente
- **D** — **DELETE** (Eliminar): Eliminar un registro

CRUD describe QUÉ queremos hacer con los datos, no cómo.

SQL

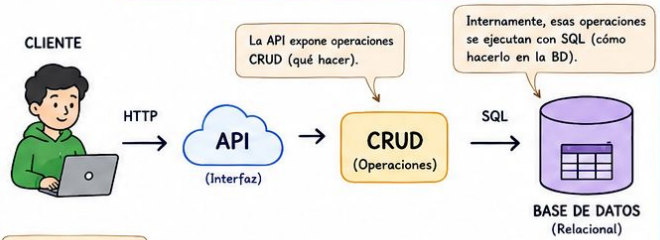
ES UN LENGUAJE DE CONSULTA

Permite comunicarnos con bases de datos relacionales para realizar operaciones.

INSERT	`INSERT INTO tabla (col1, col2) VALUES (valor1, valor2);`	(Crear)
SELECT	`SELECT * FROM tabla WHERE condición;`	(Leer)
UPDATE	`UPDATE tabla SET col = valor WHERE condición;`	(Actualizar)
DELETE	`DELETE FROM tabla WHERE condición;`	(Eliminar)

SQL es el CÓMO: el lenguaje que usamos para hablar con la base de datos.

RELACIIÓN ENTRE CLIENTE, API, CRUD Y SQL



Ejemplo completo:

- Cliente hace una petición HTTP: `DELETE /usuarios/15` (Quiere eliminar al usuario con ID 15)
- La API interpreta la operación CRUD: Operación: DELETE, Recurso: usuarios, ID: 15
- La API ejecuta SQL en la BD: `DELETE FROM usuarios WHERE id = 15;`

COMPARACIÓN RÁPIDA

ASPECTO	CRUD	SQL
Qué es	Modelo / concepto de operaciones	Lenguaje de consulta
Se enfoca en	Qué queremos hacer con los datos	Cómo interactuar con la base de datos
Operaciones	Create, Read, Update, Delete	INSERT, SELECT, UPDATE, DELETE, y muchas más
Depende de SQL	No necesariamente	Sí, se usa en BD relacionales
Dónde se usa	Diseño de APIs, lógica de negocio, sistemas	Consultas e interacción directa con la BD
Base de datos	Puede usarse con cualquier almacenamiento	Principalmente con BD relacionales (MySQL, PostgreSQL, SQL Server, etc.)

EJEMPLO PRÁCTICO: tabla usuarios

id	nombre	email
1	Ana	ana@ejemplo.com
2	Luis	luis@ejemplo.com
3	Carla	carla@ejemplo.com
...		

INTENCIÓN (CRUD) → **IMPLEMENTACIÓN (SQL)**

- C CREATE → Crear un usuario → `INSERT INTO usuarios ...`
- R READ → Obtener usuarios → `SELECT * FROM usuarios ...`
- U UPDATE → Actualizar un usuario → `UPDATE usuarios SET ... WHERE id = ...`
- D DELETE → Eliminar un usuario → `DELETE FROM usuarios WHERE id = ...`

¿CRUD siempre usa SQL?

No. CRUD es independiente del almacenamiento.
Puede implementarse sobre diferentes tecnologías.

Base de datos SQL · MongoDB (NoSQL) · Archivos (JSON, CSV) · Google Sheets · Redis

Lo importante es la abstracción:
La API ofrece operaciones CRUD, y el sistema decide cómo y dónde guardar los datos.

EN RESUMEN

CRUD = QUÉ operación queremos realizar (Modelo conceptual)
Define las 4 acciones básicas sobre los datos.

SQL = CÓMO realizamos esas operaciones en una base de datos relacional
(Lenguaje de consulta)

Juntos permiten que los sistemas guarden, consulten, modifiquen y eliminen datos de forma organizada.

EXTRACCIÓN (EXTRACT)

Obtener datos desde distintas fuentes y traerlos a nuestro sistema

FUENTES DE DATOS

APIs
Servicios web que devuelven datos (JSON)

Web
Páginas web (HTML)

OCR
Imágenes o PDFs escaneados

Archivos
CSV, Excel, JSON, TXT, etc.

Bases de datos
Relacionales o NoSQL

MÉTODOS DE EXTRACCIÓN

1. DESDE APIs (VÍA HTTP GET)

```
import requests
url = "https://maps.googleapis.com/maps/api/place/nearbysearch/json"
params = {
  "location": "-34.6037,-58.3816",
  "radius": 1000,
  "type": "restaurant",
  "key": "API_KEY"
}
respuesta = requests.get(url, params=params)
datos = respuesta.json()
```

Resultado: JSON

```
{
 "results": [
  {"name": "Café Uno", "rating": 4.5, ..},
  {"name": "Café Dos", "rating": 4.2, ..}
 ]
```

2. WEB SCRAPING (EXTRACCIÓN DE HTML)

```
import requests
from bs4 import BeautifulSoup

html = requests.get("https://ejemplo.com").text
soup = BeautifulSoup(html, "html.parser")
titulo = soup.find("h1")
print(titulo.text)
```

Resultado: Datos estructurados

titulo	fecha	autor
Ejemplo de artículo	12/05/24	Juan
Otro artículo	11/05/24	Ana
...		

3. OCR (IMÁGENES A TEXTO)

```
from PIL import Image
import pytesseract

imagen = Image.open("factura.jpg")
texto = pytesseract.image_to_string(imagen)
print(texto)
```

Resultado: Texto

```
Factura N° 000123
Fecha: 12/05/2024
Cliente: Juan Pérez
Total: $1250.00
...
```

4. DESDE ARCHIVOS (CSV, EXCEL, JSON, ETC.)

```
import pandas as pd

df_csv = pd.read_csv("ventas.csv")
df_excel = pd.read_excel("ventas.xlsx")
print(df_csv.head())
```

Resultado: Tabla de datos

fecha	producto	cantidad	precio
2024-05-12	Producto A	3	1200
2024-05-12	Producto B	1	800
...			

5. DESDE BASES DE DATOS (VÍA CONSULTAS SQL)

```
import sqlite3

conn = sqlite3.connect("empresa.db")
cursor = conn.cursor()
cursor.execute("SELECT nombre, salario FROM empleados")
datos = cursor.festhall()
```

Resultado: Registros

nombre	salario
María Gómez	85000
Carlos Ruiz	92000
Lucía Fernández	78000
...	

RESULTADO DE LA EXTRACCIÓN

Datos obtenidos y listos para la siguiente etapa del pipeline (Transformación)

CARACTERÍSTICAS

- ✓ Conectar con fuentes heterogéneas
- ✓ Obtener datos de forma automatizada mediante scripts
- ✓ Traer los datos al sistema para su uso posterior

Make automatización visual para conectar datos, apps y APIs

¿QUÉ ES MAKE?

Make es una plataforma no-code/low-code para conectar aplicaciones y automatizar procesos mediante flujos visuales llamados **scenarios.**

¿PARA QUÉ ARMAR FLUJOS?

- Automatizar tareas repetitivas sin programar todo desde cero
- Conectar sistemas que no hablan entre sí
- Mover y transformar datos entre fuentes
- Integrar APIs, IA y servicios externos
- Ahorrar tiempo y reducir errores manuales

¿Y EL MODELO FREEMIUM?

Make tiene un plan gratuito con límites de uso. Ideal para aprender y prototipar. A medida que crecen la frecuencia de ejecución, operaciones y necesidades de colaboración, existen planes pagos con más capacidades.

GRATIS

- Operaciones limitadas/mes
- Escenarios limitados
- Uuuarios limitados
- Funcionalidades básicas

PLANES PAGOS

- Más operaciones
- Más escenarios
- Más usuarios
- Funciones avanzadas
- Soporte prioritario

¿CÓMO ES UN FLUJO EN MAKE?



Los datos se mueven y se transforman paso a paso

COMPONENTES PRINCIPALES

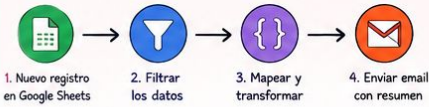
Trigger (Disparador) Inicia el flujo.	Ejemplos: Webhook, Scheduler (tiempo), Nuevo registro
Modules (Módulos) Realizan acciones con apps y servicios.	Ejemplos: Google Sheets, Gmail, Drive, HTTP/API, Slack, SQL, etc.
Filters / Routers Deciden por dónde sigue el dato.	Ejemplos: Filtro (si/entonces), Router (múltiples caminos)
Transformations Limpian y adaptan los datos.	Ejemplos: Mapear, Formatear, Convertir tipos, Agregar, Parsear JSON, etc.
Output (Salida) Deja el resultado en otro sistema.	Ejemplos: Google Sheets, BigQuery, Drive, CRM, Base de datos, API, etc.

ETL CON MAKE



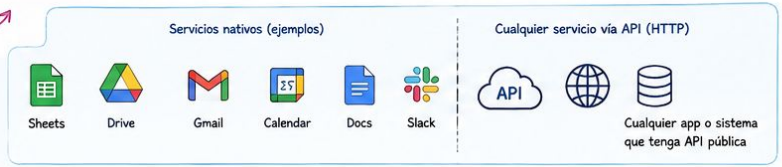
Ejemplo concreto: API de ventas → Make → normalizar datos → cruzar con Google Sheets → guardar en BigQuery

EJEMPLO VISUAL DE FLUJO



Cada círculo es un módulo. Las flechas muestran el camino que siguen los datos.

Make puede trabajar con módulos nativos de servicios como Google Sheets, Drive o Gmail, y también mediante HTTP/API para conectarse a prácticamente cualquier servicio.



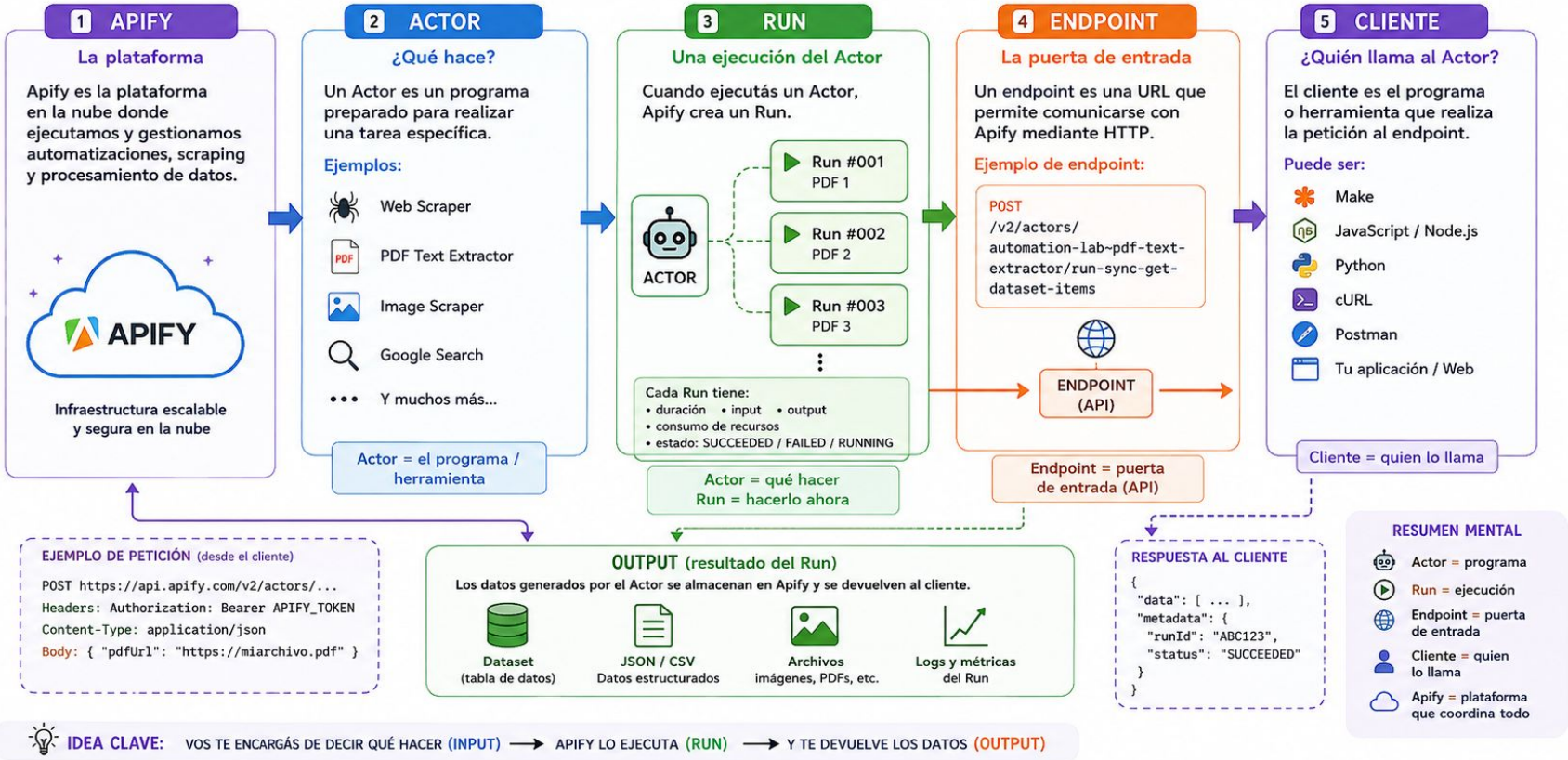
IDEA CLAVE

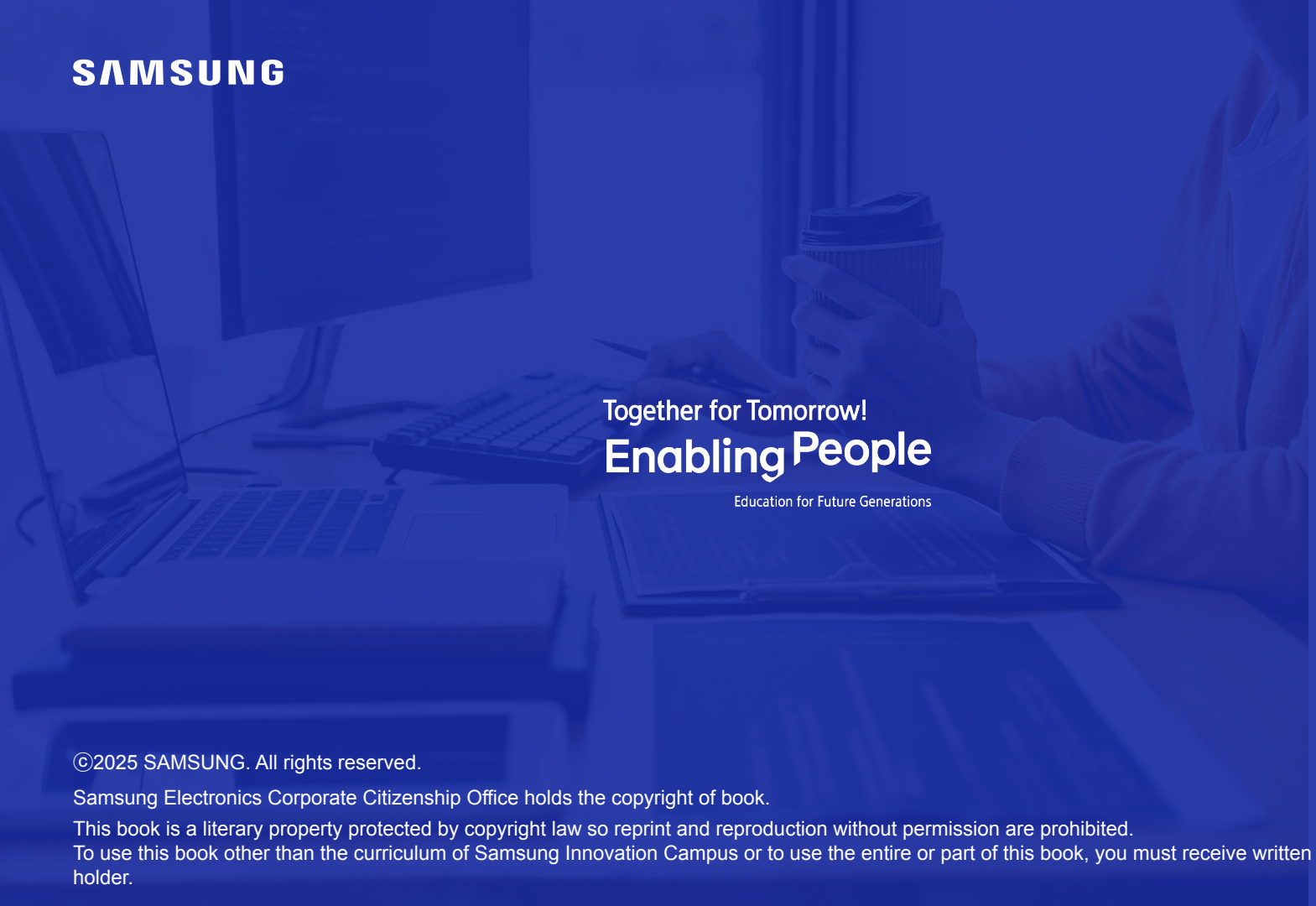
Make funciona como una capa de orquestación: recibe datos, los transforma y los lleva automáticamente de un sistema a otro.

¿CÓMO FUNCIONA APIFY?

De una idea en tu aplicación a datos listos para usar.

APIFY
Actores que trabajan por vos.





ⓒ2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.